Introduction to Databases

RELATIONAL DATABASES

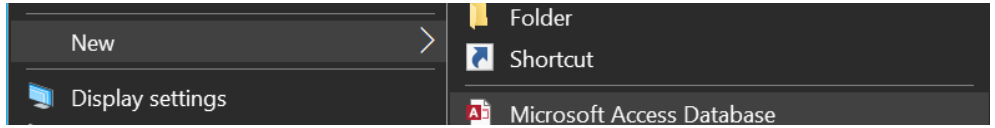
Access

MICROSOFT

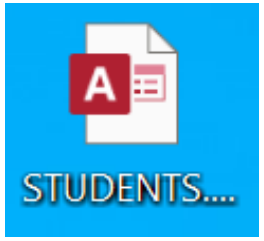
Introduction

- **Structured Query Language (SQL)** is a programming language designed to manage data stored in relational databases.

1. Right click on your preferred Folder (e.g., Desktop) and select **New → Microsoft Access Database**



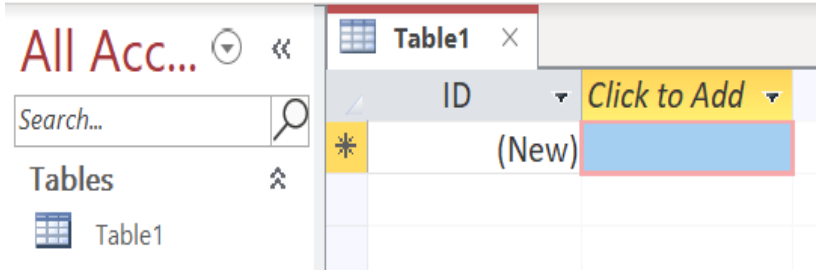
2. Rename your database (e.g., STUDENTS)



3. Open your database.

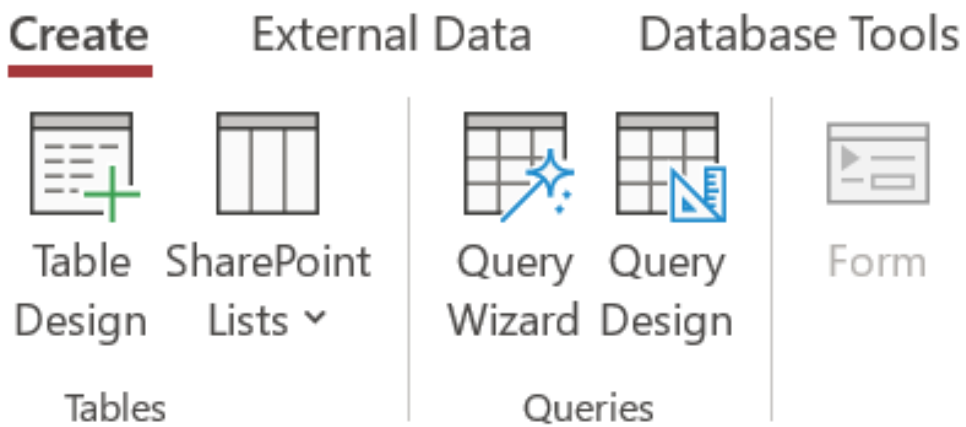
Relational Database

- A **relational database** is a type of database that organizes data into **rows** and **columns**.
- **Rows** are commonly referred to as **records** or tuples and **columns** are referred to as **fields** or attributes.
- A **key** is a column or a set of columns used to uniquely identify each row (or record) in a table and to establish relationships between different tables.

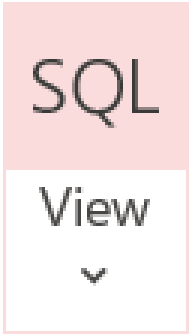


SQL View

1. Click on **Create** and select **Query Design**



2. Then click on **SQL View**



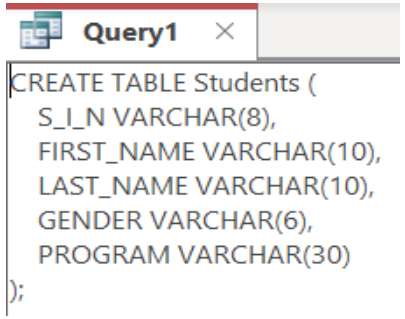
SQL Query

- **SQL queries** (also known as **SQL commands** or **SQL statements**) allow users to easily manage data in database.
- We want our SQL Query to **CREATE** the fields shown below;

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM

CREATE TABLE

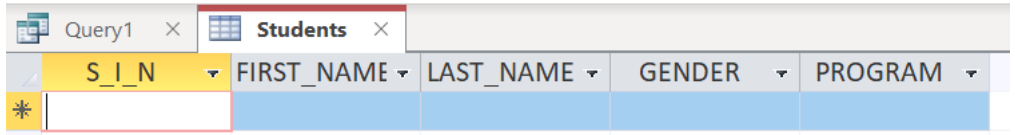
1. Write the SQL Command below;



2. Click on Run!



- Here is our desired **TABLE**



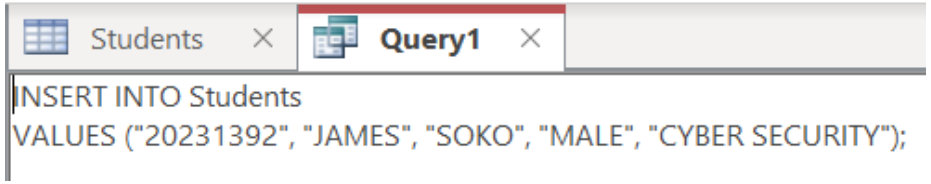

```
CREATE TABLE table_name (
 column_1 datatype,
 column_2 datatype,
 column_3 datatype
);
```

INSERT INTO

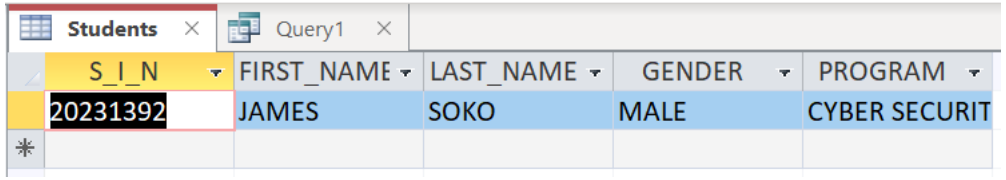
- Now are goal is to add some records to our database as shown below;

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM
20231392	JAMES	SOKO	MALE	CYBER SECURITY
20231947	EVANS	BWALYA	MALE	CYBER SECURITY
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY

1. Write the SQL Query below;



2. Select the table (Students) then click on **View → Design View → Datasheet View**



INSERT INTO (Syntax)

INSERT INTO table_name (column_1, column_2)

VALUES ('value_1', 'value_2');

Students | Query1 | Query2

```
INSERT INTO Students (S_I_N, FIRST_NAME, LAST_NAME, GENDER, PROGRAM)
VALUES ('20231947', 'EVANS', 'BWALYA', 'MALE', 'CYBER SECURITY');
```

Students | Query1 | Query2

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM
20231392	JAMES	SOKO	MALE	CYBER SECURITY
20231947	EVANS	BWALYA	MALE	CYBER SECURITY
*				

INSERT INTO table_name

VALUES ("value_1", "value_2");

Students | Query1 | Query2 | Query3

```
INSERT INTO Students
VALUES ("20231552", "FAITH", "MOONO", "FEMALE", "CYBER SECURITY");
```

students

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM
20231392	JAMES	SOKO	MALE	CYBER SECURITY
20231947	EVANS	BWALYA	MALE	CYBER SECURITY
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY

INSERT INTO – Multiple Records

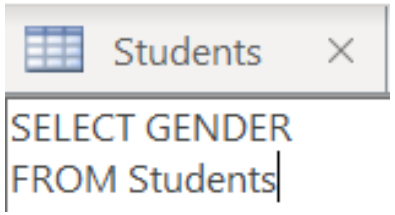
```
INSERT INTO table_name
VALUES ("column_1", "column_2"),
       ("column_1", "column_2"),
       ("column_1", "column_2"),
       ("column_1", "column_2");
```

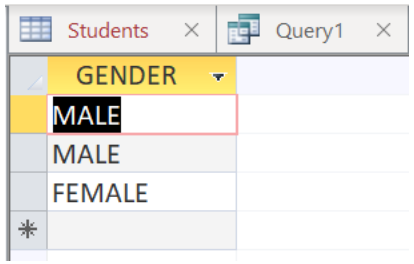
```
INSERT INTO students
VALUES (20231392, "JAMES", "SOKO", "MALE", "CYBER SECURITY"),
 (20231947, "EVANS", "BWALYA", "MALE", "CYBER SECURITY"),
 (20231552, "FAITH", "MOONO", "FEMALE", "CYBER SECURITY"),
 (20230145, "ROSARIA", "PHIRI", "FEMALE", "CYBER SECURITY");
```

SELECT

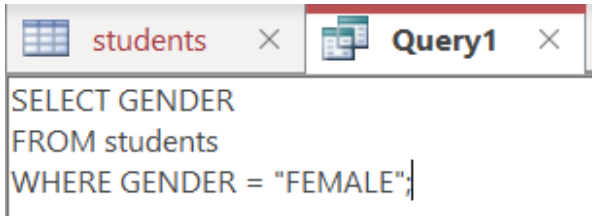
- At this point we would like to **SELECT** only female students **FROM** our database.

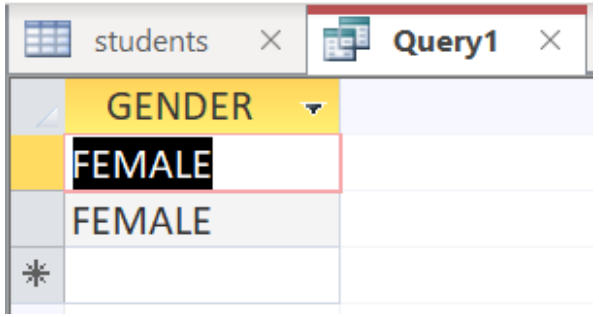
1. Here is the syntax to SELECT the entire column minus targeting something such as females





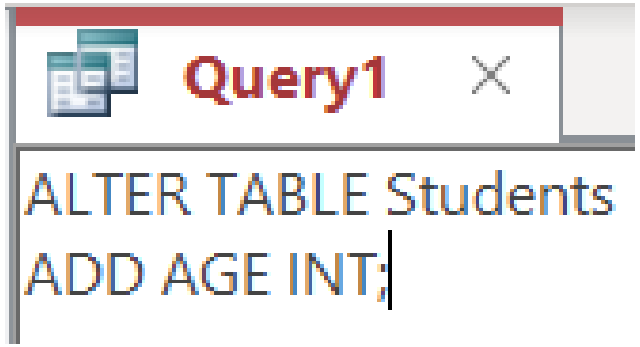
2. Now we **SELECT** GENDER **FROM** Students **WHERE** GENDER = "FEMALE";





ALTER TABLE

- After realizing that we need a column that as AGE, we advise you to **ALTER** the TABLE and **ADD** another field
- Make sure the **Students** table is not open in Datasheet View or Design View.



- Here is our updated table

students

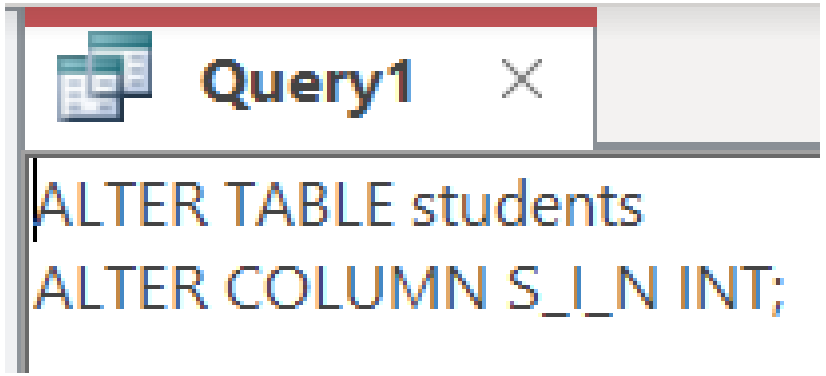
S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	
20231947	EVANS	BWALYA	MALE	CYBER SECURITY	
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY	
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY	

ALTER TABLE (Continues…)

- Let's assume we want to **DROP** the field **AGE**.
- Below is the query we can input:

```
ALTER TABLE students
DROP AGE;
```

- After realizing that the data type of the **S_I_N** and **AGE** is **VARCHAR**.
- You are tasked to **MODIFY** the datatype to **INT**.



UPDATE

- Your task is to **UPDATE** AGE field with the following

students

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	25
20231947	EVANS	BWALYA	MALE	CYBER SECURITY	24
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY	23
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY	26

- Do the same for the rest of the data

Query1

```
UPDATE students
SET AGE = 25
WHERE FIRST_NAME = "JAMES";
```

- Update statements will allow you to edit records in a table.

DELETE

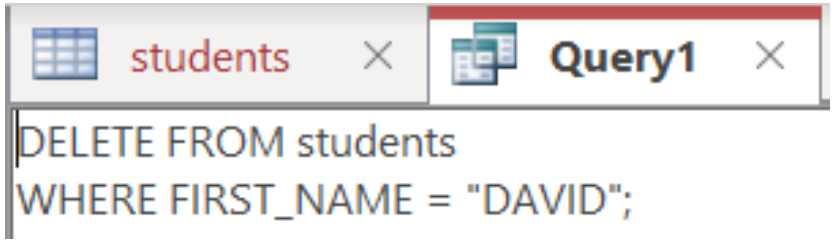
- Let's **INSERT INTO students** with **VALUES**

students

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	25
20231947	EVANS	BWALYA	MALE	CYBER SECURITY	24
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY	23
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY	26
20251010	DAVID	ZULU	MALE	DATA SCIENCE	20

- Our goal is to **DELETE FROM students** the record for David Zulu because he is not part of the Cyber Security class.

- DELETE command is used to remove data from the database.
- In this case, the entire record will be deleted.



students

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	25
20231947	EVANS	BWALYA	MALE	CYBER SECURITY	24
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY	23
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY	26
#Deleted	#Deleted	#Deleted	#Deleted	#Deleted	#Deleted

Operators

Comparison Operators

Symbol	Meaning
=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
BETWEEN	Within a range
LIKE	Pattern matching
IN	Matches any value in a list

Logical Operators

Symbol	Meaning
AND	Both conditions must be true
OR	At least one condition must be true
NOT	Negates a condition

BETWEEN

- Let's assume we are interested in students who are **BETWEEN** the AGE of 23 – 25.

students | Query1

```
SELECT *
FROM students
WHERE AGE BETWEEN 23 AND  25;
```

- In this case we used a wildcard * which means we are **SELECT**ing everything from the table.

students | Query1

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	25
20231947	EVANS	BWALYA	MALE	CYBER SECURITY	24
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY	23

- **NOTE:** All this while we where **SELECT**ing a specific field e.g., AGE and we could only see that very field.

LIKE

- To understand this query, first we need to INSERT a new record shown below.

20240001	AARON	NYAKAPANDA	MALE	CYBER SECURITY	22

students | Query1

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	25
20231947	EVANS	BWALYA	MALE	CYBER SECURITY	24
20231552	FAITH	MOONO	FEMALE	CYBER SECURITY	23
20230145	ROSARIA	PHIRI	FEMALE	CYBER SECURITY	26
20240001	AARON	NYAKAPANDA	MALE	CYBER SECURITY	22

- Now let's just filter students who started pursuing their programs in 2023.

```
SELECT *
FROM students
WHERE S_I_N LIKE '2023%';
```

Logical Operators

AND

X	Y	Z
0	0	False
0	1	False
1	0	False
1	1	True

OR

X	Y	Z
0	0	False
0	1	True
1	0	True
1	1	True

EXPRESSING	COMMENT
0=0	True
0=1	False
1=0	False
1=1	True

AND – True Statement

- Let's look for a **MALE** student who is **25** years old.

students | Query1

```
SELECT *
FROM students
WHERE GENDER = "MALE" AND AGE = 25;
```

students | Query1

S_I_N	FIRST_NAME	LAST_NAME	GENDER	PROGRAM	AGE
20231392	JAMES	SOKO	MALE	CYBER SECURITY	25

- **NOTE:** Both statements are true

students | Query1

```
SELECT *
FROM students
WHERE LAST_NAME = "SOKO" AND 1=1;
```

students | Query1

```
SELECT *
FROM students
WHERE LAST_NAME = "SOKO" AND 0=0;
```

AND – False Statement

- Notice the statements below will give us blank results:

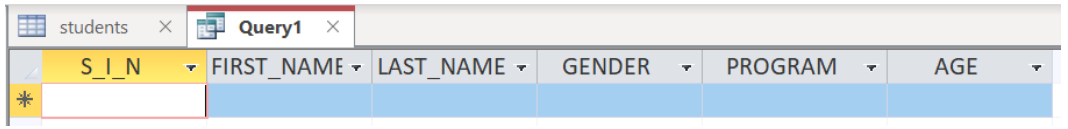
students | Query1

```
SELECT *
FROM students
WHERE LAST_NAME = "SOKO" AND 1=0;
```

students | Query1

```
SELECT *
FROM students
WHERE LAST_NAME = "SOKO" AND 0=1;
```

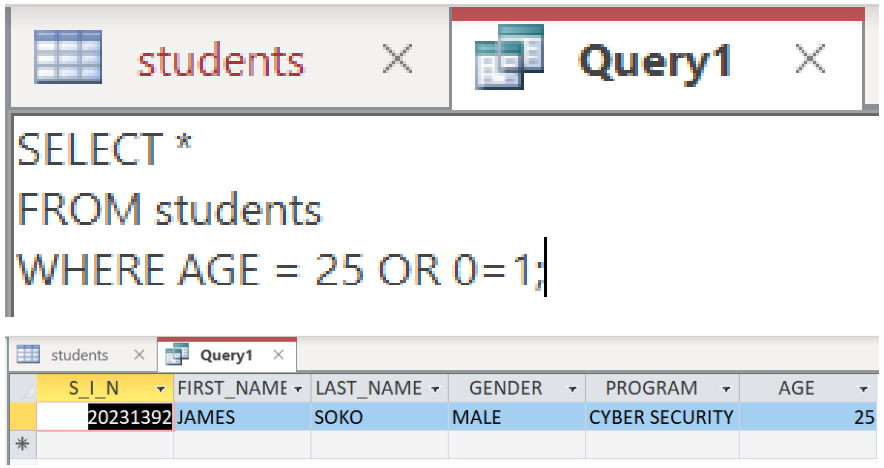
- Results:



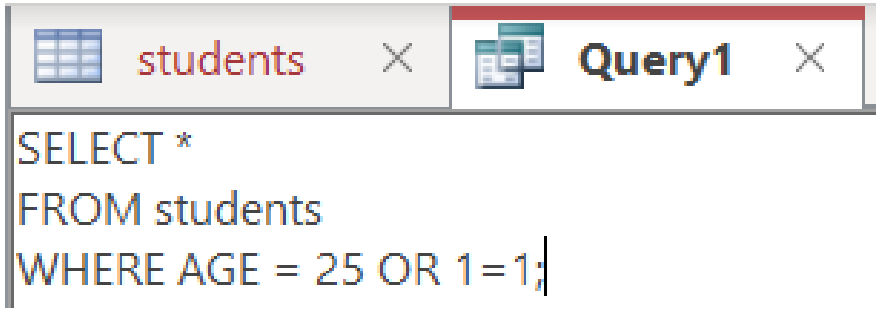
- **AND** works when both statements are true.
- **SOKO** is available in **LAST_NAME** hence true but **1=0** and **0=1** is not true.
- This results in a blank result.

OR

- Let's look for a student who is **25** years old.

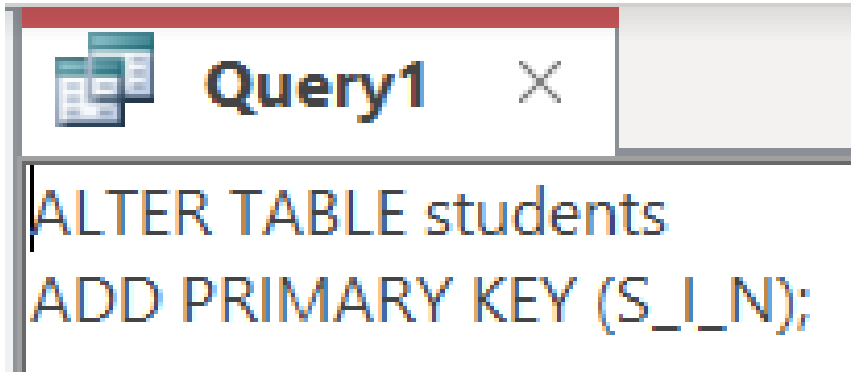


- Notice that **0=1** is false and only **25** is true, hence we are getting that result.
- If both statements become true then will have all students popping up.



PRIMARY KEY

- In our case, **S_I_N** is unique to each and every student, hence we would like to make it has our **PRIMARY KEY**.



- Otherwise, we can even do this when **CREAT**ing the **TABLE**

```
CREATE TABLE students (
  S_I_N INT PRIMARY KEY
);
```

- Each **S_I_N** must be unique and **NOT NULL** because it's a **PRIMARY KEY**.

FOREIGN KEY

- Now let's **CREATE** a new **TABLE** called **results** with the records given below:

S_I_N	CA /40	EXAM /60	TOTAL /100	GRADE
20231392	30.2	50	80.2	A
20231947	26.6	55	81.6	A
20231552	30.3	56	86.3	A
20230145	24.6	48	72.6	B+

- Here is our query:

Query1

```
CREATE TABLE results (
S_I_N INT PRIMARY KEY,
`CA /40` FLOAT,
`EXAM /60` FLOAT,
`TOTAL /100` FLOAT,
GRADE VARCHAR(2),
FOREIGN KEY(S_I_N) REFERENCES students(S_I_N)
);
```

FOREIGN KEY (Continues…)

- Now let's **INSERT** the records…

```
INSERT INTO results
VALUES (20231392, 30.2, 50, 80.2, "A"),
       (20231947, 26.6, 55, 81.6, "A"),
       (20231552, 30.3, 56, 86.3, "A"),
       (20230145, 24.6, 48, 72.6, "B+");
```

- **NOTE:** Inserting multiple **VALUES** is not supported in Microsoft Access Database.

Query1 | results

S_I_N	CA /40	EXAM /60	TOTAL /100	GRADE
20230145	24.6	48	72.6	B+
20231392	30.2	50	80.2	A
20231552	30.3	56	86.3	A
20231947	26.6	55	81.6	A

FOREIGN KEY (Continues…)

- Now, let's search the results for **JAMES SOKO** but only the **GRADE**…

Query1 | results | Query2

```
SELECT students.FIRST_NAME, students.LAST_NAME, results.GRADE
FROM students
INNER JOIN results ON students.S_I_N = results.S_I_N
WHERE students.S_I_N = 20231392;
```

Query1 | results | Query2

FIRST_NAME	LAST_NAME	GRADE
JAMES	SOKO	A

NOTICE

This PowerPoint presentation was prepared by Soko James with the purpose of covering the SQL Commands we did in class with Mr. Paul M. Mutunda. Any private information used in this handout e.g., SIN and CA was for the purpose of mimicking our course and not disclosing private info.

QUESTIONS

CYS330

Prepared by James Soko for Mr. Paul M. Mutunda ©
Kapasa Makasa University

QUESTION 1

1. With so much experience in bug bounty, analyze the two snippet codes given below.

CustomerID	Name	Password
111	Henry Mutale	3434a
222	James Soko	@0396
333	Evans Bwalya	Mafia123

- SELECT * FROM Customers WHERE CustomerID = 111 or 1=1;
- SELECT CustomerID, Name, Password FROM Customer WHERE CustomerID = 111 or 1=1;

QUESTION 1 (Continues…)

a) With the aid of an object given, explain the difference between the two lines of SQL codes. [5]

b) Briefly explain why it posses a vulnerability in a named database. [5]

c) Provide tangible solutions to the business entity using this database based on your experience in Cybersecurity related matters. [5]

QUESTION 2

Ref No.	Manufacturer	Model	Doors	0 – 100 Kph time (sec)	Top speed (kph)	Price (ZMK)
1015	Bentley	Arnage	4	6.0	250	300000
1023	Porsche	Boxster	2	5.5	266	55000
1158	Fiat	Stile	5	12.4	170	14000
1911	Maserati	GT	2	4.8	288	105000
2051	Lexus	GS	4	6.3	250	95000
2081	Renault	Laguna	5	8.4	220	25000
2516	Porsche	Cayenne	5	5.6	260	13000
3331	VW	Golf	3	9.3	200	27500
3456	Lotus	Elise	2	5.2	205	42500

QUESTION 2 – Answers

1. Will first **CREATE TABLE** cars

SQL 1

```
1  CREATE TABLE cars (
2  "Ref N0." INT,
3  Manufacturer TEXT,
4  Model TEXT,
5  Doors INT,
6  "0 - 100 Kph time (sec)" DECIMAL,
7  "Top speed (kph)" INT,
8  "Price (ZMK)" INT
9  );
```

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: cars

Filter in any co…

Ref N0.	Manufacturer	Model	Doors	0 - 100 Kph time (sec)	Top speed (kph)	Price (ZMK)
Filter	Filter	Filter	Filter	Filter	Filter	Filter

QUESTION 2 – Answers (Continues…)

2. **INSERT INTO cars** the **VALUES** given in the table

SQL 1

```
1  INSERT INTO cars
2  VALUES (1015, "Bently", "Arnage", 4, 6.0, 250, 300000),
3         (1023, "Porsche", "Boxter", 2, 5.5, 266, 55000),
4         (1158, "Fiat", "Stile", 5, 12.4, 170, 14000),
5         (1911, "Maserati", "GT", 2, 4.8, 288, 105000),
6         (2051, "Lexus", "GS", 4, 6.3, 250, 95000),
7         (2081, "Renault", "Laguna", 5, 8.4, 220, 25000),
8         (2516, "Porsche", "Cayenne", 5, 5.6, 260, 13000),
9         (3331, "VW", "Golf", 3, 9.3, 200, 27500),
10        (3456, "Lotus", "Elise", 2, 5.2, 205, 42500);
```

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: cars

	Ref N0.	Manufacturer	Model	Doors	0 - 100 Kph time (sec)	Top speed (kph)	Price (ZMK
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1015	Bently	Arnage	4	6	250	30000
2	1023	Porsche	Boxter	2	5.5	266	5500
3	1158	Fiat	Stile	5	12.4	170	1400
4	1911	Maserati	GT	2	4.8	288	10500
5	2051	Lexus	GS	4	6.3	250	9500
6	2081	Renault	Laguna	5	8.4	220	2500
7	2516	Porsche	Cayenne	5	5.6	260	1300
8	3331	VW	Golf	3	9.3	200	2750
9	3456	Lotus	Elise	2	5.2	205	4250

QUESTION 2 (Continues…)

a) How many records are shown in the section of the database above?

b) How many fields are shown in the section part of this database?

c) If the following query was input

(Top Speed (kph) > 250 OR (Doors = 2)

Using Ref No. only, which items would be outputs?

d) Write down a query which outputs cars which costs more than K60000 and have a 0 – 100 kph time (sec) of less than 7.0

QUESTION 2 – Answers (Continues…)

c) Hence the answer is:

i. 1023

ii. 1911

iii. 2516

iv. 3456

SQL 1 ×

```
1  SELECT *
2  FROM cars
3  WHERE "Top speed (kph)" > 250 OR Doors = 2;
```

	Ref N0.	Manufacturer	Model	Doors	0 - 100 Kph time (sec)	Top speed (kph)	Price (ZMK)
1	1023	Porsche	Boxter	2	5.5	266	5500(
2	1911	Maserati	GT	2	4.8	288	10500(
3	2516	Porsche	Cayenne	5	5.6	260	1300(
4	3456	Lotus	Elise	2	5.2	205	4250(

QUESTION 2 – Answers (Continues…)

d) The query is

SQL 1

```
1  SELECT *
2  FROM cars
3  WHERE "Price (ZMK)" > 60000 AND "0 - 100 Kph time (sec)" < 7.0;
```

	Ref N0.	Manufacturer	Model	Doors	0 - 100 Kph time (sec)	Top speed (kph)	Price (ZMK)
1	1015	Bently	Arnage	4	6	250	300000
2	1911	Maserati	GT	2	4.8	288	105000
3	2051	Lexus	GS	4	6.3	250	95000